

## Parte II

1. Relembre (da Parte I) que uma representação possível de polinómios (de expoentes positivos) é pela sequência dos coeficientes.

```
type Pol = [Float]
```

Têm que se armazenar também os coeficientes nulos pois será a posição do coeficiente na lista que dará o grau do monómio. Por exemplo:  $[7,0,1,0,0,4]$  corresponde ao polinómio  $7 + x^2 + 4x^5$ .

- (a) Defina a função `soma :: Pol -> Pol -> Pol` que soma dois polinómios.
- (b) Defina a função `mult :: Pol -> Pol -> Pol` que multiplica dois polinómios.

2. Considere o seguinte tipo para representar expressões proposicionais:

```
data Prop = Var String | Not Prop | And Prop Prop | Or Prop Prop
```

```
p1 :: Prop
```

```
p1 = Not (Or (And (Not (Var "A")) (Var "B")) (Var "C"))
```

- (a) Declare `Prop` como instância da classe `Show`, de forma a que a expressão `p1` seja apresentada da seguinte forma:  $\neg((\neg A \wedge B) \vee C)$
- (b) Defina a função `eval :: [(String, Bool)] -> Prop -> Bool`, que dada uma *valoração* (i.e., uma correspondência entre variáveis proposicionais e valores booleanos) calcula o valor lógico de uma expressão proposicional. Por exemplo,  
`eval [("B", True), ("C", False), ("A", True)] p1 == True` e  
`eval [("A", True), ("C", True), ("B", False)] p1 == False`.
- (c) Uma proposição diz-se na *forma normal negativa* se as negações só estão aplicadas às variáveis proposicionais. Por exemplo, a proposição  $((A \vee \neg B) \wedge \neg C)$  está na forma normal negativa, enquanto `p1` não está.

Defina a função `nnf :: Prop -> Prop` que recebe uma proposição e produz uma outra que lhe é equivalente, mas que está na forma normal negativa. Por exemplo, o resultado de `nnf p1` deverá ser a proposição  $((A \vee \neg B) \wedge \neg C)$ .

Lembre-se das seguintes leis:  $\neg\neg A = A$ ,  $\neg(A \vee B) = \neg A \wedge \neg B$  e  $\neg(A \wedge B) = \neg A \vee \neg B$ .

- (d) Defina a função `avalia :: Prop -> IO Bool` que, dada uma proposição, pergunta ao utilizador a valoração de cada variável presente na proposição e calcula com essa informação o seu valor lógico.

*Avail :: Prop -> IO*

*h = Var d = "Indique o valor da variável:"  
 ++ show d  
 a <- getLine*

*calculaValor :: Prop -> Bool*

*eval :: [(String, Bool)] -> Prop -> Bool*

*Pergunta: IO  
 valor*

*Valor :: IO*

*Valor =*

5

*Valor :: (String, Bool) -> Prop -> IO*

*h = And e = ...  
 h = And*

*Pergunta: IO*

*numerovar :: Prop -> Int  
 numerovar [c] = 0  
 numerovar (a:b:c) =  
 1 + numerovar b*